

Saarathi Decision Engine

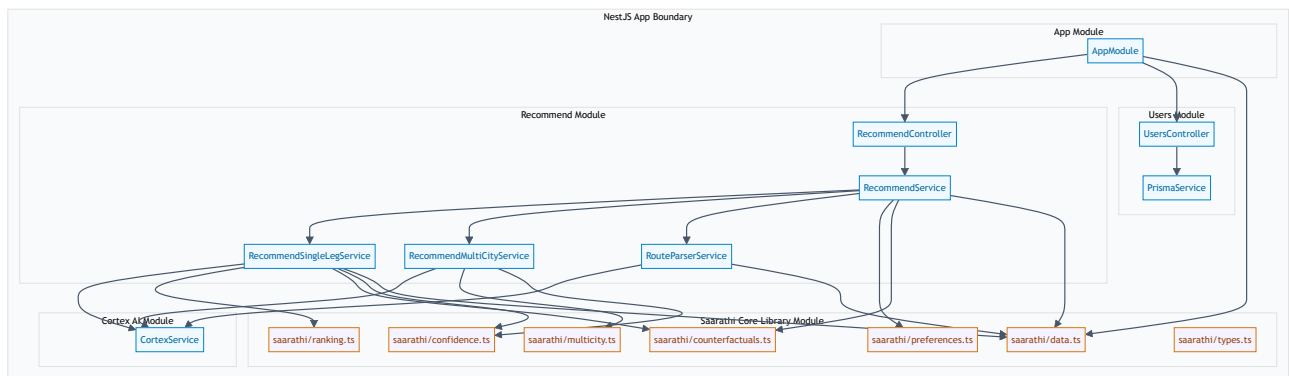
System Architecture & Workflow Specification

Document Version: 1.0.0 | Date: July 2026 | Expedia Group

1. System Module Architecture

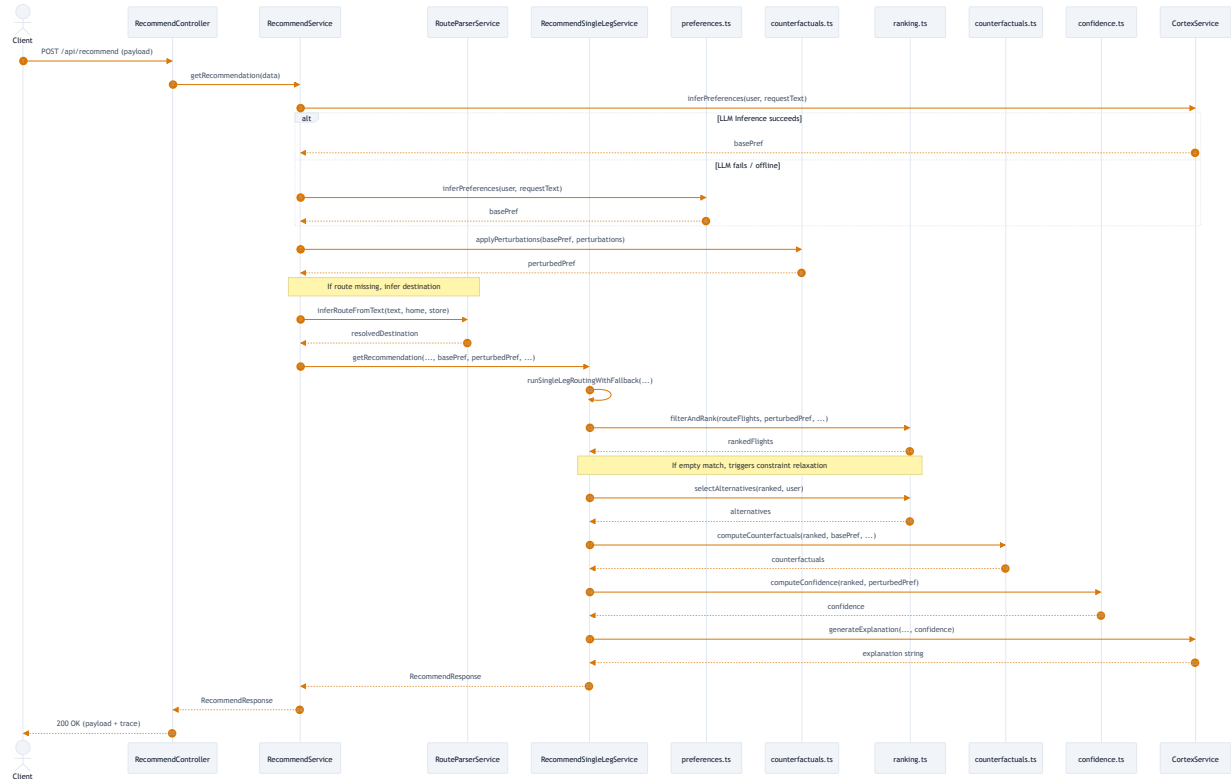
The application follows a modular NestJS dependency injection architecture integrated with a framework-free, highly testable core business library. The backend modules are broken down as follows:

- **AppModule:** The root entry point coordinating bootstrapping and cache warming.
- **RecommendModule:** Manages user requests, orchestrates leg evaluation, and parses natural language travel intentions.
- **UsersModule:** Handles traveler profile data, DB access, and active traveler queries.
- **CortexModule:** Encapsulates LLM queries and cosine-similarity check logic.
- **Saarathi Core Library (Framework-free):** Houses raw mathematical scoring models, date calculators, layover constraints, and backtracking route sequences.



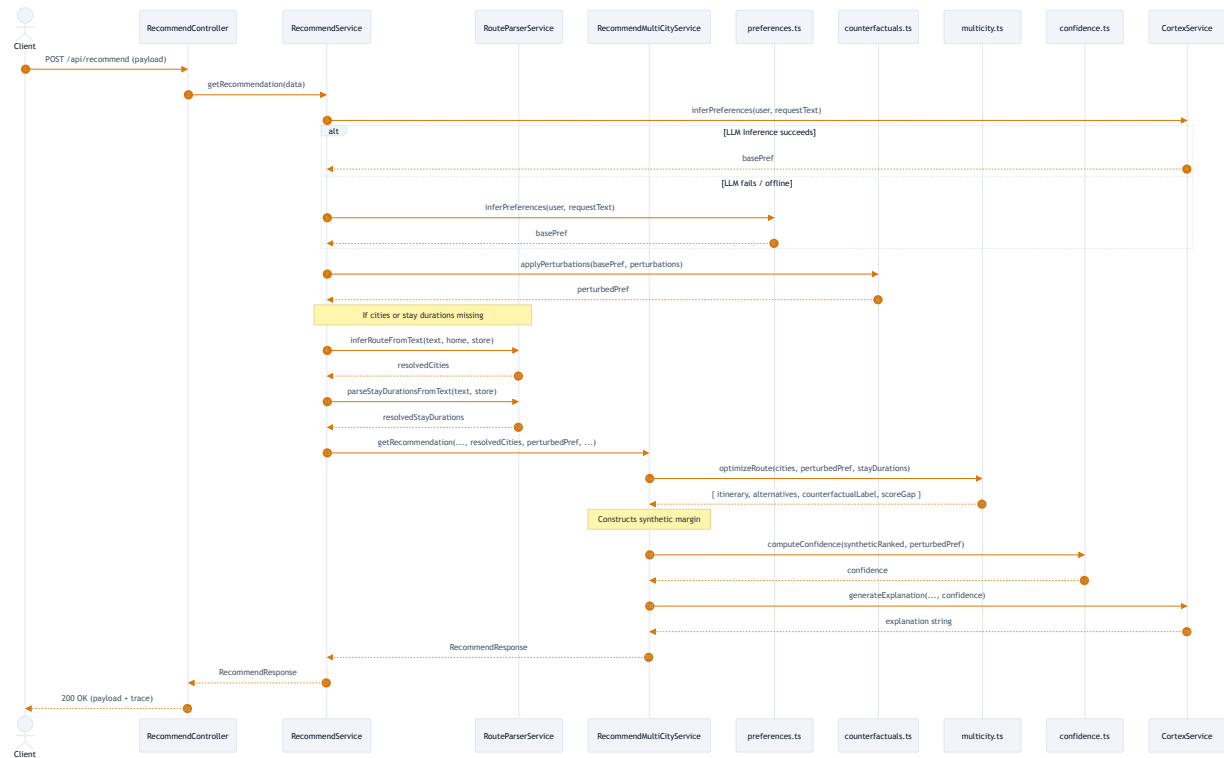
2. Request Sequence Lifecycle: Single-Leg Route

This sequence diagram documents the end-to-end lifecycle of a single-leg flight recommendation search. The controller intercepts the payload, triggers asynchronous preference weights modeling (with an offline fallback to cosine similarity checks), resolves missing destinations, filters the active flight pool, performs counterfactual inversion to find decision thresholds, and packages the technical explainability trace.



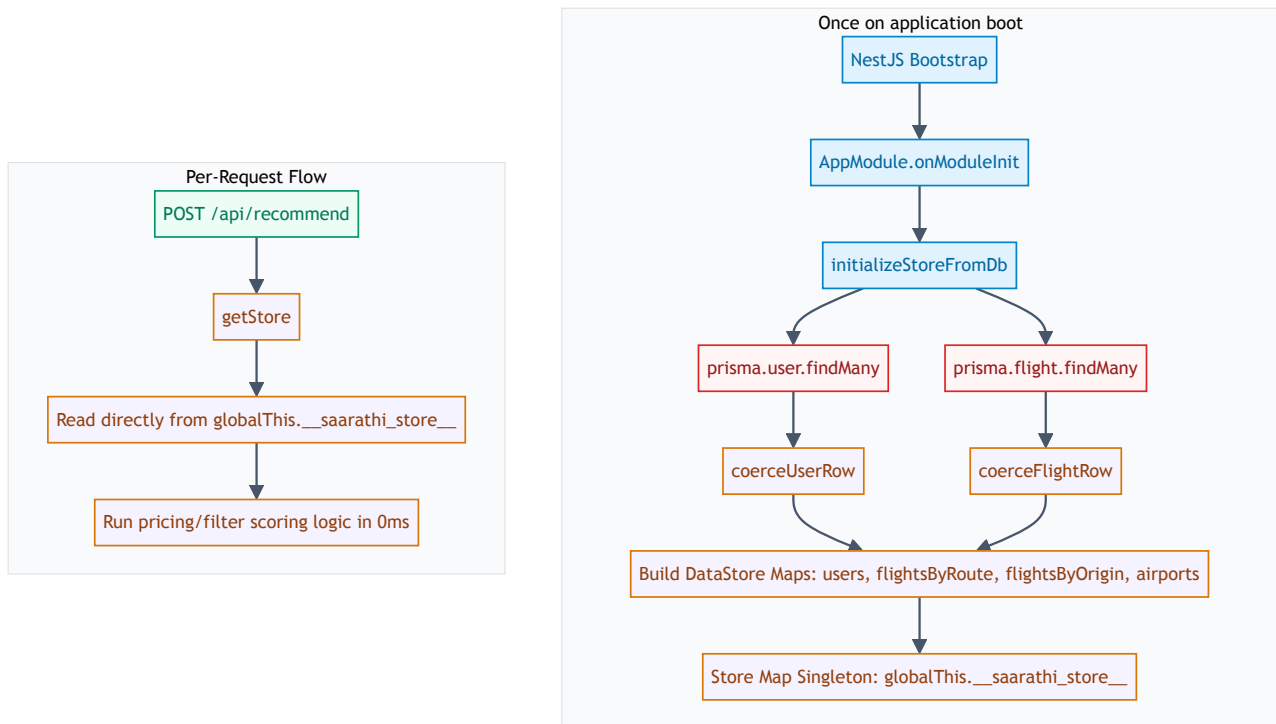
3. Request Sequence Lifecycle: Multi-City Route

The multi-city lifecycle utilizes a specialized route optimizer module. Permutations of destinations are analyzed, stay requirements are checked against departure dates, and the best sequence is computed using recursive backtracking pruned by precalculated remaining max potential scores.



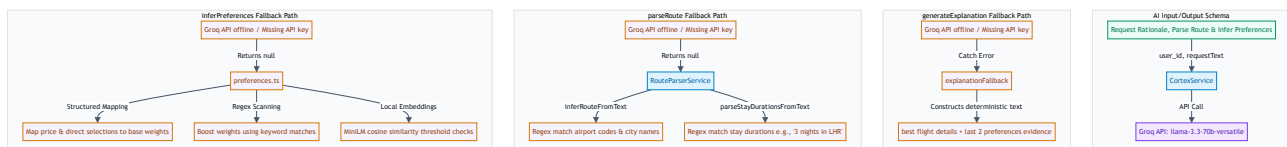
4. Boot Lifecycle & Memory Caching

To maintain sub-millisecond route calculations, the backend avoids making database queries during the evaluation of flight recommendation requests. On application boot, the database is parsed and cached into memory once.



5. AI/LLM Boundaries & Resilient Fallbacks

AI integration is designed with resilience in mind. If the external LLM provider encounters API rate limits or connection failures, local deterministic fallbacks step in, preserving full functionality:



6. SQLite & Prisma Database Schema Design

The application persists structural data inside a SQLite file. Below is the active layout and query access frequency chart:



7. Core Deterministic Routing Algorithms

✦ A. Preference Weight Inference

Calculates numeric weights (w_i in $[0, 1]$) representing a traveler's priorities: price sensitivity, direct flight preference, cabin tier, and airline loyalty. The weights are constructed by checking dropdown variables, searching free text using regex keywords, and checking semantic similarity with the local `all-MiniLM-L6-v2` embedding pipeline (cosine threshold ≥ 0.35).

✦ B. Flight Filtering & Linear Scoring

Candidate flights are evaluated using a deterministic linear equation. First, hard constraints are applied (e.g. max layover limit, dates flexible window, red-eye filtering). Then, surviving flights are scored using the normalized weights and properties:

```
Score = (costWeight * PriceScore + directWeight * DirectScore + (1 - costWeight) * 0.5 * DurationScore + convenienceWeight * 0.3 * CabinScore + 0.2 * AirlineScore + baggageWeight * BaggageScore) * DemandAdjustment * HolidayAdjustment + DayBonus
```

Where `PriceScore` and `DurationScore` are dynamically scaled to $[0, 1]$ relative to the minimum and maximum values present in the current flight pool.

✦ C. Multi-City Routing Optimizer

For multi-city requests, the optimizer generates all permutations of the destination city order. For each permutation, a recursive backtracking search is performed to match flights chronological to dates. The search is optimized via a branch-and-bound pruning step:

```
if (currentAccumulatedScore + maxPotentialRemainingScore ≤ bestScoreFound) {  
  pruneBranch(); // Skip scanning deep legs  
}
```

✦ D. Counterfactual Inversion

To locate decision boundaries for runners-up, the linear scoring function is algebraically inverted to find the exact price threshold that would make it equal the champion's score:

```
priceDrop = (Score_champion - Score_challenger) * PriceRange / (costWeight * demandMultiplier * holidayMultiplier)
```

If the required drop is $\leq 60\%$ of the flight's original price, it is returned as a realistic counterfactual threshold to present to the traveler.

✦ E. Confidence Assessment

Assigns a confidence tier (High, Medium, Low) to a recommendation. It computes the match percentage (ratio of champion score to theoretical maximum score). It outputs `High` confidence only if the score margin between the

champion and the runner-up is > 0.1 , and reduces the confidence tier to **Low** if the margin is < 0.04 or if conflicting preferences (e.g. high price sensitivity but strong premium cabin preference) are detected.